Name : Manaswi Kabadi
Class : D15C
Roll no. : 27
Batch : B

EXP - 8

Aim:

Design and train a Convolutional Neural Network (CNN) on the Fashion-MNIST image dataset using Keras Tuner for hyperparameter optimization.

Dataset Description:

The Fashion-MNIST Image Dataset used in this experiment is a built-in dataset provided by the Keras deep learning library. It is a standard benchmark dataset for image classification that contains small grayscale images of 10 fashion product categories. The dataset is preprocessed, labeled, and packaged inside Keras, allowing direct loading without manual downloading. It is a supervised learning dataset where each 28×28 grayscale image is paired with a correct class label, making it ideal for training Convolutional Neural Networks (CNNs) and evaluating classification accuracy. Due to its balanced classes, standardized format, and moderate dataset size, Fashion-MNIST is widely used as a more challenging drop-in replacement for MNIST in deep learning experiments and academic research.

- **File Type:** Built-in Keras dataset (NumPy arrays loaded via TensorFlow/Keras)
- **Dataset Size:** 70,000 images (60,000 training + 10,000 testing)
- **Image Dimension:** 28 × 28 Grayscale pixels (single channel)
- **Number of Classes:** 10 fashion categories (T-shirt/top, Trouser, Pullover, Dress, Coat, Sandal, Shirt, Sneaker, Bag, Ankle boot)

Dataset Source: https://keras.io/api/datasets/fashion_mnist/

Column Name	Data Type	Description
Image Pixels	Numerical (Integer 0–255 per grayscale channel)	28×28 grayscale pixel intensity values representing the clothing item image.
Label	Numerical (Integer 0–9)	True clothing category corresponding to each image.

Convolutional Neural Network (CNN)

Theory:

Convolutional Neural Networks (CNNs) are a class of deep learning models specifically designed for processing structured grid-like data such as images. Unlike traditional Artificial Neural Networks, CNNs automatically learn spatial hierarchies of features through convolution operations. They exploit local

connectivity, parameter sharing, and translation invariance, making them highly effective for image classification, object detection, and pattern recognition tasks.

A CNN consists of multiple layers including convolutional layers, activation layers, pooling layers, and fully connected layers. The convolution operation extracts important visual features such as edges, textures, and shapes, which are progressively combined to form high-level representations.

Let an input image be represented as $X \in R^{(H \times W \times C)}$, where H = height, W = width, and C = number of channels (C = 1 for grayscale Fashion-MNIST). Each neuron computes a weighted sum of inputs followed by an activation function:

$$Z(i, j) = \Sigma \Sigma W(m, n) X(i+m, j+n) + b$$

where W = convolution kernel (weights), k×k = kernel size, b = bias term, and Z(i, j) = feature map output.

Limitations:

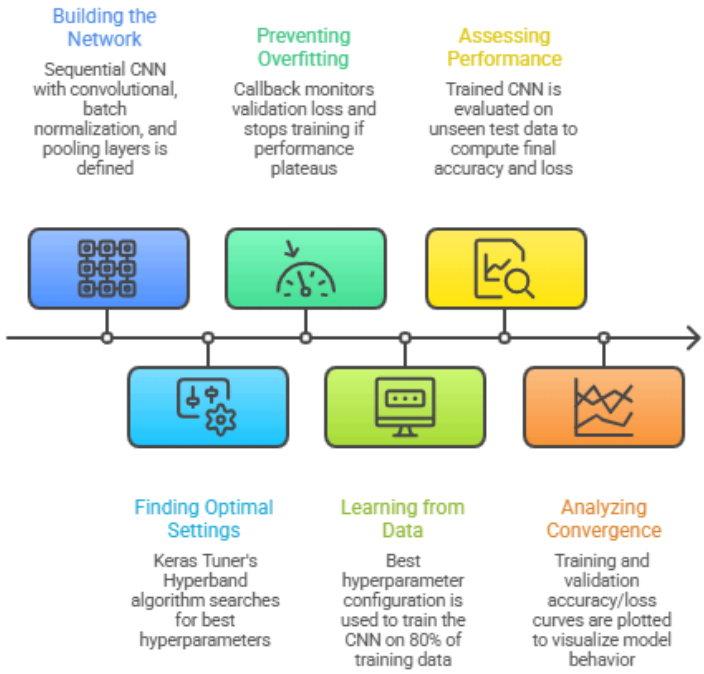
- **Requirement of Large Labeled Datasets:** CNNs require a large volume of labeled image data to achieve strong generalization. When training data is limited, CNNs tend to overfit by memorizing patterns instead of learning meaningful features.
- **High Computational and Hardware Cost:** CNNs involve millions of parameters and repeated convolution operations, making training computationally intensive. Deep CNN architectures require powerful GPUs or TPUs, large memory capacity, and long training time.
- **Black-Box Nature and Limited Interpretability:** CNNs automatically learn hierarchical features that are difficult for humans to interpret. This lack of transparency becomes a critical issue in sensitive applications such as medical diagnosis and autonomous driving.

Workflow:

- **Dataset Loading:** The built-in Keras Fashion-MNIST dataset is loaded using TensorFlow. It contains 60,000 training images and 10,000 testing images of fashion products across 10 categories. Each image is a 28×28 grayscale image.
- **Data Preprocessing:** All pixel values are normalized from the range 0–255 to 0–1 by dividing by 255. Images are reshaped to add the channel dimension (28×28×1). This normalization improves training stability and helps the CNN converge faster.
- **CNN Model Architecture Definition:** A Sequential Convolutional Neural Network is defined with multiple convolutional blocks. Each block includes Conv2D layers for feature extraction, Batch Normalization for stable learning, and MaxPooling layers for spatial reduction. The feature maps are flattened and passed to dense layers with dropout for classification into 10 classes using a Softmax output layer.
- **Hyperparameter Tuning (Keras Tuner):** The Hyperband algorithm automatically searches for optimal hyperparameters such as the number of convolution filters, dense layer size, dropout

rate, optional third convolution block, and learning rate. The objective is to maximize validation accuracy.

- **Early Stopping Strategy:** An early stopping callback monitors validation loss and halts training if performance stops improving. This reduces overfitting and saves computational resources.
- **Model Training:** The best hyperparameter configuration is used to rebuild the CNN. The model is trained for extended epochs using 80% of the training data while reserving 20% for validation.
- **Model Evaluation:** The trained CNN is evaluated on the unseen test dataset to compute final accuracy and loss, confirming how well the model performs on new data.
- **Performance Visualization:** Training and validation accuracy and loss curves are plotted across epochs to analyze convergence behavior and detect overfitting or underfitting.



Code :

```
# STEP 0: Install Keras Tuner
!pip install keras-tuner -q
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
import keras_tuner as kt
import matplotlib.pyplot as plt
import numpy as np
print("Loading Fashion-MNIST
Dataset...")
(x_train, y_train), (x_test, y_test) =
keras.datasets.fashion_mnist.load_data)
# Fashion-MNIST is grayscale (28x28x1),
reshape to add channel dimension
x_train = x_train.reshape(-1, 28, 28,
1).astype('float32') / 255.0
x_test  = x_test.reshape(-1, 28, 28,
1).astype('float32') / 255.0
print(f"Training data shape:
{x_train.shape}")
print(f"Test data shape:
{x_test.shape}")
class_names = ["T-shirt/top",
"Trouser", "Pullover", "Dress", "Coat",
               "Sandal", "Shirt",
"Sneaker", "Bag", "Ankle boot"]
plt.figure(figsize=(12, 4))
for i in range(10):
```

```
    plt.subplot(2, 5, i+1)
    plt.imshow(x_train[i].reshape(28,
28), cmap='gray')
plt.title(class_names[y_train[i][0]] if
y_train[i].ndim > 0 else
class_names[y_train[i]])
    plt.axis('off')
plt.suptitle("Sample Fashion-MNIST
Images")
plt.tight_layout()
plt.show()
def build_model(hp):
    model = keras.Sequential()
    model.add(layers.Input(shape=(28,
28, 1)))
    # Block 1
    hp_filters_1 = hp.Int('filters_1',
min_value=32, max_value=128, step=32)

model.add(layers.Conv2D(filters=hp_filt
ers_1, kernel_size=3,
activation='relu', padding='same'))

model.add(layers.BatchNormalization())

model.add(layers.MaxPooling2D(pool_size
=(2, 2)))

    # Block 2
    hp_filters_2 = hp.Int('filters_2',
min_value=64, max_value=256, step=32)

model.add(layers.Conv2D(filters=hp_filt
ers_2, kernel_size=3,
activation='relu', padding='same'))

model.add(layers.BatchNormalization())

model.add(layers.MaxPooling2D(pool_size
=(2, 2)))
    # Optional Block 3
    if hp.Boolean('conv_block_3'):
        hp_filters_3 =
hp.Int('filters_3', min_value=64,
max_value=256, step=32)

model.add(layers.Conv2D(filters=hp_filt
ers_3, kernel_size=3,
activation='relu', padding='same'))

model.add(layers.BatchNormalization())
        # No MaxPooling here - image is
already 7x7 after two pooling layers
    # Classifier Head
    model.add(layers.Flatten())
    hp_units = hp.Int('units',
min_value=128, max_value=512, step=64)

model.add(layers.Dense(units=hp_units,
activation='relu'))
    hp_dropout = hp.Float('dropout',
min_value=0.2, max_value=0.5, step=0.1)

model.add(layers.Dropout(rate=hp_dropou
t))
    model.add(layers.Dense(10,
activation='softmax'))
    # Compile
    hp_learning_rate =
hp.Choice('learning_rate',
values=[1e-2, 1e-3, 1e-4])
    model.compile(

optimizer=keras.optimizers.Adam(learnin
g_rate=hp_learning_rate),

loss='sparse_categorical_crossentropy',
        metrics=['accuracy']
    )
    return model
tuner = kt.Hyperband(
    build_model,
    objective='val_accuracy',
    max_epochs=10,
    factor=3,
    directory='fashion_tuner_dir',
```

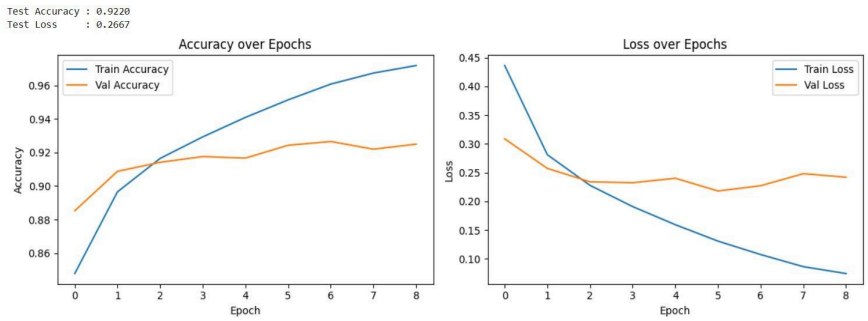
```
    project_name='fashion_mnist_cnn')
stop_early =
keras.callbacks.EarlyStopping(monitor='
val_loss', patience=3)
print("\nStarting Hyperparameter
Search...")
tuner.search(x_train, y_train,
epochs=10, validation_split=0.2,
callbacks=[stop_early])
best_hps =
tuner.get_best_hyperparameters(num_tria
ls=1)[0]
print(f"""
--- TUNING COMPLETE ---
Optimal Hyperparameters:
  - Filters Block 1   :
{best_hps.get('filters_1')}
  - Filters Block 2   :
{best_hps.get('filters_2')}
  - Use Block 3       :
{best_hps.get('conv_block_3')}
  - Dense Units       :
{best_hps.get('units')}
  - Dropout Rate      :
{best_hps.get('dropout')}
  - Learning Rate     :
{best_hps.get('learning_rate')}""")
print("\nBuilding and training the best
model...")
model =
tuner.hypermodel.build(best_hps)
history = model.fit(
    x_train, y_train,
    epochs=10,
    validation_split=0.2,
    callbacks=[stop_early])
print("\nEvaluating on Test Set...")
test_loss, test_acc =
model.evaluate(x_test, y_test)
print(f"Test Accuracy :
{test_acc:.4f}")
print(f"Test Loss     :
{test_loss:.4f}")
plt.figure(figsize=(12, 4))
plt.subplot(1, 2, 1)
plt.plot(history.history['accuracy'],
label='Train Accuracy')
plt.plot(history.history['val_accuracy'
], label='Val Accuracy')
plt.title('Accuracy over Epochs')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()

plt.subplot(1, 2, 2)
plt.plot(history.history['loss'],
label='Train Loss')
plt.plot(history.history['val_loss'],
label='Val Loss')
plt.title('Loss over Epochs')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()
plt.tight_layout()
plt.show()
pred_probs  = model.predict(x_test)
pred_labels = np.argmax(pred_probs,
axis=1)
true_labels = y_test.flatten()
plt.figure(figsize=(12, 12))
for i in range(16):
    idx = np.random.randint(0,
len(x_test))
    plt.subplot(4, 4, i+1)
    plt.imshow(x_test[idx].reshape(28,
28), cmap='gray')
    color = "green" if pred_labels[idx]
== true_labels[idx] else "red"
    plt.title(f"P:
{class_names[pred_labels[idx]]}\nT:
{class_names[true_labels[idx]]}",
color=color, fontsize=8)
    plt.axis('off')
plt.suptitle("Random Predictions (Green
= Correct, Red = Wrong)")
plt.tight_layout()
plt.show()
```

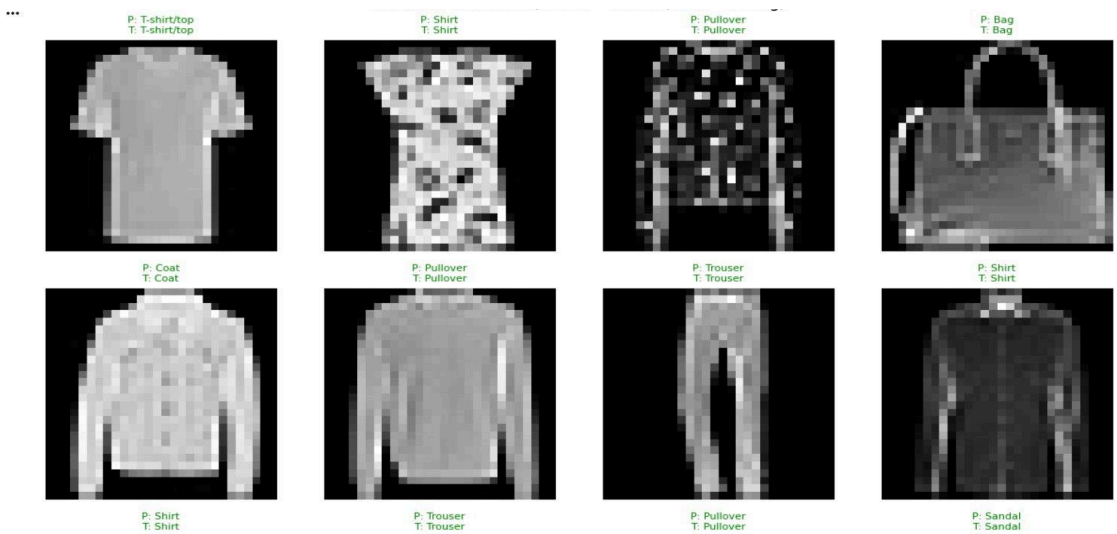
```
from sklearn.metrics import
confusion_matrix
import seaborn as sns
cm = confusion_matrix(true_labels,
pred_labels)
plt.figure(figsize=(10, 8))
sns.heatmap(cm, annot=True, fmt="d",
cmap="Blues",
            xticklabels=class_names,
yticklabels=class_names)
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.show()
class_accuracy = cm.diagonal() /
cm.sum(axis=1)
plt.figure(figsize=(10, 5))
plt.bar(class_names, class_accuracy,
color='steelblue')
plt.ylim(0, 1)
plt.ylabel("Accuracy")
plt.title("Class-wise Accuracy")
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.show()
misclassified = np.where(pred_labels !=
true_labels)[0]
plt.figure(figsize=(12, 6))
for i, idx in
enumerate(misclassified[:10]):
    plt.subplot(2, 5, i+1)
    plt.imshow(x_test[idx].reshape(28,
28), cmap='gray')
    plt.title(f"P:
{class_names[pred_labels[idx]]}\nT:
{class_names[true_labels[idx]]}",
fontsize=7)
    plt.axis('off')
plt.suptitle("Top 10 Misclassified
Images")
plt.tight_layout()
plt.show()
confidence = np.max(pred_probs, axis=1)
wrong_confidence =
confidence[misclassified]
top_wrong =
misclassified[np.argsort(-wrong_confide
nce)[:5]]

plt.figure(figsize=(10, 4))
for i, idx in enumerate(top_wrong):
    plt.subplot(1, 5, i+1)
    plt.imshow(x_test[idx].reshape(28,
28), cmap='gray')
plt.title(f"{class_names[pred_labels[id
x]]}\n({confidence[idx]:.2f})",
fontsize=8)
    plt.axis('off')
plt.suptitle("Most Confident Wrong
Predictions")
plt.tight_layout()
plt.show()
print("\n========== FINAL SUMMARY
==========")
print(f"Dataset         :
Fashion-MNIST")
print(f"Train Samples   :
{x_train.shape[0]}")
print(f"Test Samples    :
{x_test.shape[0]}")
print(f"Image Shape     :
{x_train.shape[1:]}")
print(f"Num Classes     : 10")
print(f"Best Filters B1 :
{best_hps.get('filters_1')}")
print(f"Best Filters B2 :
{best_hps.get('filters_2')}")
print(f"Block 3 Used    :
{best_hps.get('conv_block_3')}")
print(f"Dense Units     :
{best_hps.get('units')}")
print(f"Dropout Rate    :
{best_hps.get('dropout')}")
print(f"Learning Rate   :
{best_hps.get('learning_rate')}")
```

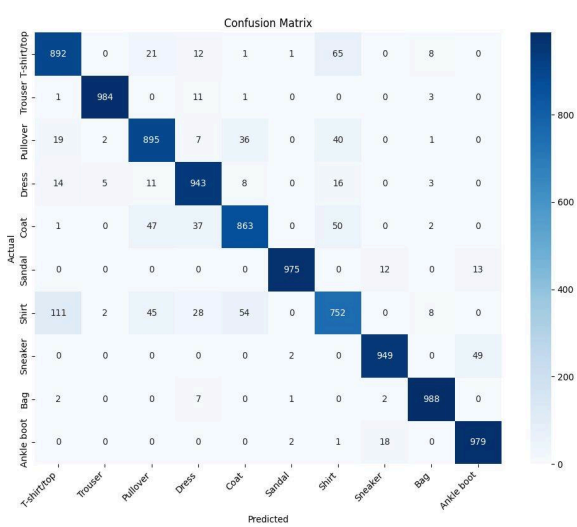
```
print(f"Test Accuracy    :
{test_acc:.4f}")
```

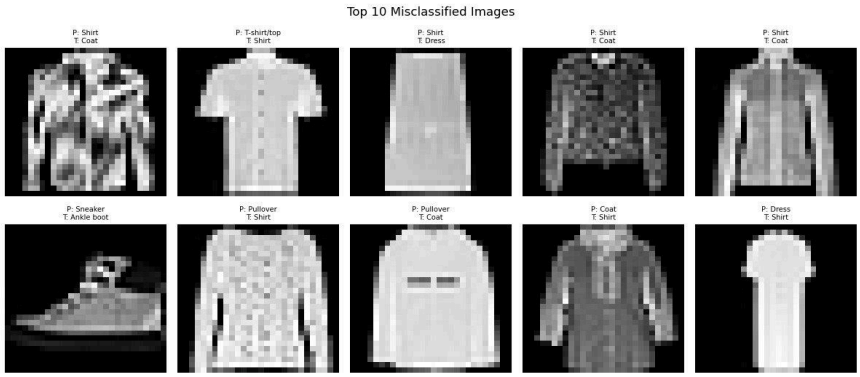
```
print(f"Test Loss        :
{test_loss:.4f}")
print("================================
====")
```

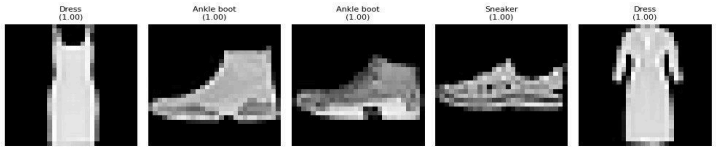
Output:












```
========== FINAL SUMMARY ==========
Dataset          : Fashion-MNIST
Train Samples    : 60000
Test Samples     : 10000
Image Shape      : (28, 28, 1)
Num Classes      : 10
Best Filters B1  : 64
Best Filters B2  : 128
Block 3 Used     : False
Dense Units      : 384
Dropout Rate     : 0.3000000000000004
Learning Rate    : 0.0001
Test Accuracy    : 0.9220
Test Loss        : 0.2667
```

Performance Analysis:

The performance of the hyperparameter-tuned Convolutional Neural Network is evaluated using validation accuracy, test accuracy, learning curves, and confusion matrix analysis. Since Fashion-MNIST is a supervised multi-class image dataset, evaluation focuses on classification accuracy, generalization ability, and class-wise prediction behavior.

- **Hyperparameter Optimization Results:** The Hyperband tuner explored multiple CNN configurations. The optimal architecture uses 64 filters in the first convolution block, 128 filters in the second block, no third convolution block, a dense layer with 384 neurons, a dropout rate of 0.3, and a learning rate of 0.0001. These settings provide a strong balance between model capacity and regularization.
- **Training vs Validation Accuracy Behavior:** The learning curves show continuous improvement in training accuracy, reaching above 97%, while validation accuracy stabilizes around 92%. The growing gap between training and validation curves suggests mild overfitting after several epochs. Early stopping helps reduce excessive over-training.
- **Test Set Performance:** Evaluation on the unseen test dataset produces a final accuracy of 92.20% and a test loss of 0.2667, closely matching validation performance. This consistency indicates that the model generalizes well and does not rely purely on memorization. For a CNN trained on grayscale 28×28 images, this accuracy is excellent for Fashion-MNIST.

- **Confusion Matrix Analysis:** The confusion matrix shows strong performance for structured categories such as Trouser, Bag, Sandal, and Ankle boot, which have distinct shapes. More errors occur in visually similar categories such as Shirt, Coat, and Pullover, where overlapping features make discrimination difficult.

Hyperparameter Tuning:

Convolutional Neural Networks are highly sensitive to hyperparameter choices. Automated tuning is performed using Keras Tuner with the Hyperband algorithm. The tuned hyperparameters in this experiment are:

- **Number of Convolution Filters:** The number of filters in each convolution block is tuned to control feature extraction capacity. Block 1 searches between 32–128 filters, Block 2 between 64–256 filters. The optimal model selects 64 filters in Block 1 and 128 filters in Block 2.
- **Optional Third Convolution Block:** The tuner decides whether adding an extra convolution block improves performance. The best configuration does not include the third block, indicating that a two-block architecture suffices for the simpler 28×28 grayscale images of Fashion-MNIST.
- **Dense Layer Size:** The fully connected layer is tuned between 128–512 neurons in steps of 64. The optimal configuration selects 384 neurons, providing sufficient representation power after convolutional feature extraction.
- **Dropout Regularization Rate:** Dropout is tuned between 0.2–0.5 to reduce overfitting. The tuner selects a 0.3 dropout rate, balancing regularization with information retention.
- **Learning Rate:** The learning rate is tuned from [0.01, 0.001, 0.0001]. The optimal value is 0.0001, enabling stable and precise convergence without overshooting.

Conclusion:

This experiment successfully demonstrates the design, tuning, and evaluation of a Convolutional Neural Network on the Fashion-MNIST dataset. The hyperparameter-tuned CNN achieves a test accuracy of 92.20%, demonstrating excellent learning and generalization on grayscale fashion images. The two-block CNN architecture with 64 and 128 filters, 384 dense neurons, 0.3 dropout, and a learning rate of 0.0001 proved to be the optimal configuration identified by the Hyperband tuner. Performance analysis shows that the model captures meaningful feature hierarchies across all 10 clothing categories while maintaining consistent validation and test accuracy. Visualization tools such as learning curves and confusion matrices provide deeper insight into class-wise behavior and model limitations — notably, the model struggles most with visually similar categories like Shirt, Coat, and Pullover. Overall, the experiment highlights the practical workflow of modern CNN training, hyperparameter tuning, and evaluation in real-world image classification tasks.